Manual de Usuario

Equipo de Prácticas de Control de Motor CC

Departamento de Ingeniería de Sistemas y Automática (ISA)

Universidad de Sevilla · ialvarado@us.es

November 6, 2026

En qué consiste este equipo: un motor DC Maxon con encoder magnético de 14 bits, controlado en lazo cerrado desde MATLAB/Simulink a través de un Arduino. El lazo de control *corre íntegramente en el PC*: Arduino solo lee el encoder y aplica el PWM que MATLAB le ordena cada **10 ms** (100 Hz). La comunicación es por USB/serie a 115 200 baudios.

Lo que necesita el profesor: conectar el USB, identificar el puerto serie en MATLAB, y lanzar el experimento. El resto es automático.

Contents

1 Descripción del equipo **3**
- 1.1 Componentes principales . . . 3
- 1.2 Esquema de conexiones . . . 3
- 1.3 Parámetros del sistema . . . 3

2 Puesta en marcha inicial (una sola vez) **3**
- 2.1 Instalar las librerías de Arduino . . . 4
- 2.2 Cargar el firmware en el Arduino . . . 4
- 2.3 Comprobar el cableado de la placa shield . . . 4

3 Uso en cada sesión de prácticas **5**
- 3.1 Paso 1 — Conectar el hardware . . . 5
- 3.2 Paso 2 — Identificar el puerto serie en MATLAB . . . 5
- 3.3 Paso 3 — Lanzar el experimento desde MATLAB . . . 5
- 3.4 Paso 4 — Guardar los datos del experimento . . . 6

4 Referencia rápida **6**
- 4.1 Protocolo de comunicación serie . . . 6
- 4.2 Pines del Arduino / Shield . . . 6

4.3 Parámetros de `config.m` 6
4.4 Conversión señal de control → tensión 7

5 Resolución de problemas frecuentes **7**

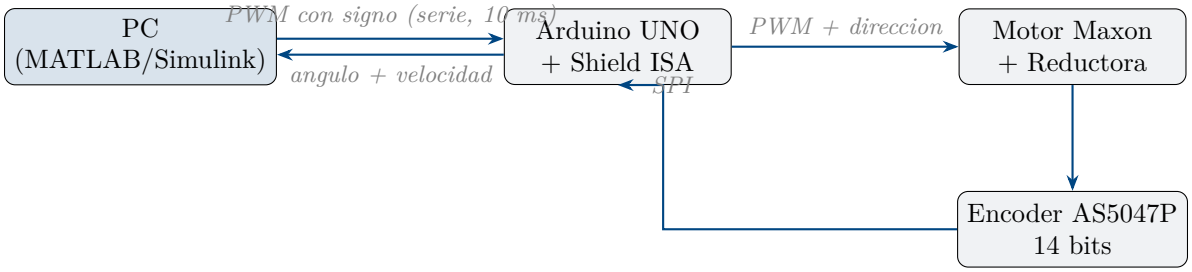
6 Repositorio y recursos **8**

1 Descripción del equipo

1.1 Componentes principales

Componente	Modelo	Función
Motor	Maxon A-max 22	Motor DC de precisión con reductora planetaria GP 22 B
Encoder	ams AS5047P	Encoder magnético 14 bits (16 384 pasos/vuelta), montado en el lado de carga
Driver	TB6612FNG	Driver de motor dual integrado en la placa shield
Microcontrolador	Arduino UNO	Lee el encoder, aplica PWM y se comunica con MATLAB por USB
Placa shield	ISA custom	Apilable sobre el Arduino; integra driver + conector encoder

1.2 Esquema de conexiones



Nota

El lazo de control completo reside en MATLAB. Arduino actúa únicamente como **interfaz hardware**: lee el encoder por SPI, calcula la velocidad como diferencia de posición dividida por $T_s = 0.01$ s, y aplica la señal de control al driver.

1.3 Parámetros del sistema

Parámetro	Valor	Notas
Periodo de muestreo T_s	10 ms (100 Hz)	Fijado en el firmware de Arduino
Velocidad de comunicación	115 200 baudios	USB/serie
Rango señal de control	$u \in [-255, 255]$	Equivale a ± 12 V
Resolución encoder	≈ 0.022ř/paso	16 384 pasos/vuelta
Modelo aproximado	$G(s) = K/(\tau s + 1)$	$K \approx 3$, $\tau \approx 0.07$ s a 12 V

2 Puesta en marcha inicial (una sola vez)

Esta sección solo es necesaria la **primera vez** que se configura el equipo, o si se cambia el Arduino. No hace falta repetirla en cada sesión de prácticas.

2.1 Instalar las librerías de Arduino

1. Abrir **Arduino IDE 2.x**.
2. Ir a `Herramientas → Gestor de librerías`.
3. Buscar e instalar `AS5047P` (autor: Marginally Clever Robots).
4. Buscar e instalar `MsTimer2` (autor: Javier Valencia).

■ Nota

Estas librerías solo se instalan una vez en el PC del laboratorio. Si se cambia de ordenador, hay que repetir este paso.

2.2 Cargar el firmware en el Arduino

1. Conectar el Arduino al PC por USB.
2. Abrir el archivo `firmware/arduino/main/main.ino` del repositorio.
3. Seleccionar la placa correcta: `Herramientas → Placa → Arduino UNO`.
4. Seleccionar el puerto: `Herramientas → Puerto → COMx` (Windows) o `/dev/tty.usbmodem...` (Mac/Linux).
5. Pulsar el botón ↑ **Subir** (o `Ctrl+U`).
6. Esperar a que aparezca el mensaje *"Subida exitosa"*.

✔ Correcto

Para verificar que el firmware funciona: abrir el **Monitor Serie** del IDE (`Ctrl+Shift+M`), configurarlo a `115200 baudios`. Deben aparecer caracteres binarios (datos en crudo) a 100 Hz. Si no aparece nada, revisar el cableado del encoder.

△ Atención

El firmware ya está cargado en el equipo del laboratorio. Solo hay que volver a cargarlo si se reemplaza el Arduino o se actualiza el firmware.

2.3 Comprobar el cableado de la placa shield

La placa shield se apila directamente sobre el Arduino (conexión sin cables). Verificar que:

- La shield está bien asentada sobre el Arduino (todos los pines encajados).
- El cable del motor está conectado a los bornes del driver.
- El cable del encoder (conector JST de 7 pines) está conectado a la shield.
- La alimentación de 12 V está conectada al borne de la shield.

3 Uso en cada sesión de prácticas

Este es el procedimiento habitual al inicio de cada sesión. Con el equipo ya configurado, tarda menos de dos minutos.

3.1 Paso 1 — Conectar el hardware

1. Conectar el cable USB del Arduino al PC.
2. Conectar la fuente de alimentación de 12 V al equipo (borne de la shield).
3. El LED del Arduino debe encenderse. ✔

△ Atención

Conectar siempre primero el USB y *después* la alimentación de 12 V. Al desconectar, en orden inverso: primero la alimentación, luego el USB.

3.2 Paso 2 — Identificar el puerto serie en MATLAB

1. Abrir MATLAB.
2. Ejecutar en la ventana de comandos:

```
serialportlist("available")
```

MATLAB devolverá algo parecido a:

```
ans =
  "/dev/tty.usbmodem1101"     % Mac
  "COM5"                      % Windows
```

3. Anotar el nombre del puerto del Arduino (normalmente el único nuevo que aparece al conectar el USB).

■ Nota

En el archivo `software/matlab/config.m` hay un parámetro `PORT` que hay que actualizar con este valor si es diferente al configurado. El modelo Simulink `MotorCC_V2.slx` detecta el puerto automáticamente en Mac/Linux; en Windows puede ser necesario configurarlo manualmente en ese archivo.

3.3 Paso 3 — Lanzar el experimento desde MATLAB

Opción A: Script MATLAB (identificación / control manual)

1. Abrir `software/matlab/config.m` y verificar/ajustar `PORT`.
2. Ejecutar el script correspondiente:

```
run('config.m')            % Cargar parametros globales
run('identificacion.m')    % Experimento de identificacion por escalon
% o bien:
run('main_control.m')      % Lazo de control en tiempo real
```

Opción B: Modelo Simulink

1. Abrir `software/matlab/MotorCC_V2.slx`.
2. Pulsar el botón **Run** (▶) en la barra de herramientas de Simulink.
3. El modelo se conecta automáticamente al Arduino y comienza a ejecutar el lazo de control a 100 Hz.

✔ Correcto

Si todo va bien, en el scope de Simulink se verán en tiempo real la posición angular (en grados) y la señal de control aplicada. El motor responderá a la referencia configurada.

3.4 Paso 4 — Guardar los datos del experimento

Los datos se almacenan automáticamente en el workspace de MATLAB. Al terminar el experimento, ejecutar:

```
run('plot_results.m')    % Visualizar los resultados
save('resultados_lab01.mat', 't', 'angulo', 'velocidad', 'u')   % Guardar
```

4 Referencia rápida

4.1 Protocolo de comunicación serie

Dirección	Formato	Significado
Arduino → MATLAB	`'V'` | `float`(4B) | `float`(4B) | `float`(4B) | `'\n'`	ángulo [°], velocidad [°/s], PW
MATLAB → Arduino	`'A'` | `int16`(2B) | `'\n'`	señal de control $u \in [-255, 25$

Frecuencia de envío: **100 Hz** ($T_s = 10$ ms). Formato binario (little-endian).

4.2 Pines del Arduino / Shield

Pin Arduino	Señal	Descripción
4	CS (SPI)	Chip Select encoder AS5047P
11	MOSI	SPI MOSI
12	MISO	SPI MISO
13	CLK	SPI Clock
9	EnA (PWM)	Enable / PWM canal A del driver
7	Ain1	Dirección motor (bit 1)
8	Ain2	Dirección motor (bit 2)

4.3 Parámetros de `config.m`

```
PORT  = '/dev/tty.usbmodem1101';  % Puerto serie (ajustar segun equipo)
BAUD  = 115200;                    % Velocidad de comunicacion [baud]
Ts    = 0.01;                      % Periodo de muestreo [s]
```

```
Kp    = 1.0;                    % Ganancia proporcional
Ki    = 0.5;                    % Ganancia integral
Kd    = 0.0;                    % Ganancia derivativa
T_exp = 10;                     % Duracion del experimento [s]
```

4.4 Conversión señal de control → tensión

La señal de control $u \in [-255, 255]$ se mapea linealmente a la tensión del motor:

$$V_{\text{motor}} = u \cdot \frac{12\,\text{V}}{255} \approx u \cdot 0.047\,\text{V}$$

Ejemplos: $u = 255 \Rightarrow 12$ V, $u = 128 \Rightarrow 6$ V, $u = 0 \Rightarrow$ motor parado.

5 Resolución de problemas frecuentes

✘ El puerto serie no aparece en MATLAB

- Comprobar que el cable USB está bien conectado.
- En Windows: abrir el Administrador de dispositivos y buscar *"Puertos COM y LPT"*. Si aparece con un triángulo amarillo, reinstalar el driver CH340 o CP2102.
- En Mac: ejecutar `ls /dev/tty.usb*` en Terminal.
- Reiniciar MATLAB después de conectar el Arduino.

✘ El motor no gira

- Verificar que la alimentación de 12 V está encendida.
- Comprobar que la señal de control enviada desde MATLAB no es 0.
- Revisar que los cables del motor están bien conectados al driver.
- Si la señal es muy pequeña ($|u| < 30$) el motor puede no vencer la zona muerta (fricción estática). Es normal; se aborda en la práctica Lab 02.

✘ El encoder no responde / posición siempre 0

- Comprobar el conector JST del encoder (7 pines): debe estar completamente encajado.
- Abrir el Monitor Serie del Arduino IDE a 115200 baudios: si no salen datos, el encoder no se inicializa. El firmware imprime un mensaje de error en ese caso.
- Verificar que el imán del eje está correctamente alineado sobre el encoder (distancia de aire recomendada: 0.5 – 1.5 mm).

✘ Simulink se desconecta a mitad del experimento

- Suele ocurrir si hay otro programa usando el mismo puerto serie (p.ej., el Monitor Serie del Arduino IDE). **Cerrar el Monitor Serie antes de lanzar Simulink.**
- Si el problema persiste, desconectar y reconectar el USB, y relanzar el modelo.

✘ La señal de velocidad es muy ruidosa

- Es normal: la velocidad se obtiene como diferencia de posición dividida por T_s, lo que amplifica el ruido cuantificado del encoder.
- Solución: aplicar un filtro paso bajo de primer orden en MATLAB/Simulink con constante de tiempo $T_f \approx 0.02 - 0.05$ s. Se practica en Lab 01.

6 Repositorio y recursos

Recurso	Ubicación
Repositorio completo	`github.com/DocMuerde/Motor-CC-Control-Lab`
Firmware Arduino	`firmware/arduino/main/main.ino`
Scripts MATLAB	`software/matlab/`
Modelo Simulink	`software/matlab/MotorCC_V2.slx`
Guía de prácticas Lab 01	`labs/lab01/lab01_isa.pdf`
Guía de prácticas Lab 02	`labs/lab02/lab02_isa.pdf`
Planos PCB	`hardware/pcb/`
Piezas 3D	`hardware/3d-printing/`

■ Nota

Para clonar el repositorio completo:

```
git clone https://github.com/DocMuerde/Motor-CC-Control-Lab.git
```